

based on the file type

- Argument *d* indicates the directory file type
- In addition to the directory, the *find* command supports the following file types:

- b: block device
- c: character device
- f: regular file
- l: symbolic links
- p: named pipe
- s: socket

The *find* command allows you to perform a search based on the file size. We can provide an *EXPRESSION* which will compare files that are greater than, less than or equal to the provided size. We can also perform a search based on permissions, for which we have to use the *-perm* option. For instance, the command below searches for files with the 664 permission:

```
$ find src-dir -type f -perm 664
```

The *find* command allows you to perform some additional operations while searching. For instance, it provides the *-delete* option, which will remove a file that matches with *EXPRESSION*. The following example shows the usage of the *-delete* option:

```
$ find src-dir -type f -size +2k -delete
```

In the above example, all files greater than 2KB in size will be deleted.

We can also execute bash commands while performing a search. We can achieve this using the *-exec* option.

2) diff

We often need to compare the contents of files. Doing this manually is a tiresome and error-prone task. But fortunately, GNU/Linux provides a command for this, which will compare files, line by line, and report if any differences are found. When the *diff* command is combined with the *patch* command, it makes a powerful combination. With this command, we can apply changes from one file to the other. This section describes both these commands.

First, create two files with the following content:

```
# file-1: version1.txt
str1
str2
str3
```

```
# file-2: version2.txt
str1
str3
```

Now compare these files using the *-u* option, which stands for unified *diff*:

```
$ diff -u version1.txt version2.txt
--- version1.txt    2017-12-30 14:06:38.120849370 +0530
+++ version2.txt    2017-12-30 14:06:46.976750148 +0530
@@ -1,3 +1,2 @@
 str1
-str2
 str3
```

The above output shows that the line 'str2' is not present in the *version2.txt* file. We can store this *diff* output in a file and apply it as a patch. To create a patch file, just redirect the output to some file as shown below:

```
$ diff -u version1.txt version2.txt > diff.patch
```

If we apply this patch to the *version1.txt* file, then it will remove the 'str2' line from this file. The example below shows this:

```
$ patch -p1 version1.txt < diff.patch
patching file version1.txt
```

```
$ diff -u version1.txt version2.txt
```

After applying the patch, both files will be identical; hence, the *diff* command does not show any differences here.

To revert the patch, execute the commands given below and follow the on-screen instructions shown below:

```
$ patch -p1 version1.txt < diff.patch
patching file version1.txt
Reversed (or previously applied) patch detected! Assume -R?
[n] y
```

The combination of *diff* and *patch* is really powerful. Many version control systems like Git, Subversion and CVS use this feature.

3) rename

Renaming multiple files is one of the common tasks of a sysadmin. GNU/Linux provides the *rename* command which will serve our purpose. It is particularly useful when we want to rename multiple files with a specific pattern. For instance, the command below renames all *.TXT* files to *.txt*:

```
$ rename 's|.TXT|.txt|' *
```

Storing similar types of files in a directory is also a very common task. We can do it very easily with a combination of *find* and the *mv* command. The command below moves all MP3 files to a *target-dir* directory: